

第6课 质量属性：性能

北京交通大学软件学院

严禁复制



目录



01. 定义

02. 战术

03. 案例

04. 决策

严禁复制



01 复制
禁止

Amazon开放日：极致性能

- Amazon Prime Day

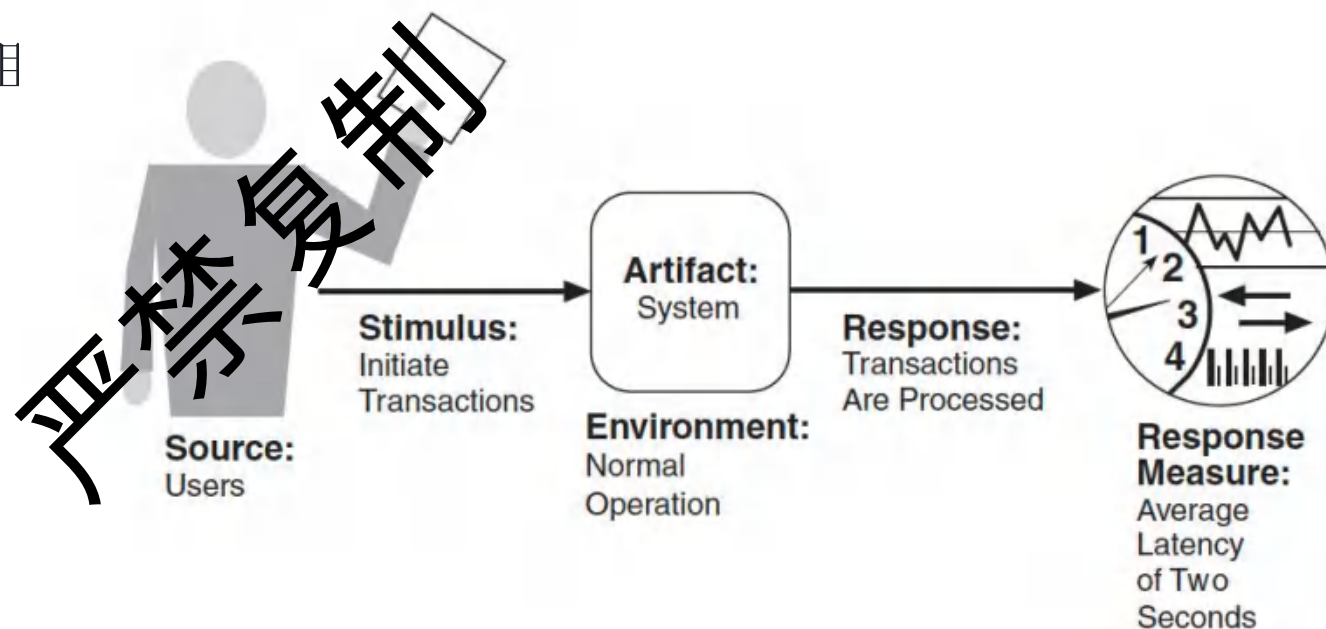
- 2019年，1.5亿会员参与，销售额超过70亿美元，商品超过1.75亿件。
- 2020年，会员数增长至超过1.75亿，104亿美元，商品超过2.5亿件。
- 2021年，到2亿以上，约112亿美元，商品数量超过3亿件。
- 2022年，约2.2亿，约127亿美元，售出商品超过3.5亿件。
- 2023年，超过2.4亿，突破140亿美元，接近3.8亿件。

- Amazon DynamoDB

- 订单的核心处理环节，如购物车更新、库存锁定、订单生成等，平均6毫秒内完成响应；
- 稳定性高达99.999%；
- 2020年峰值达到8010万次请求每秒；

性能的度量

1. 源：系统内部或外部（用户或者定时器）
2. 刺激：周期性、偶发性或随机事件的到来
3. 工件：系统或系统中的一个或多个组
4. 环境：
 1. 操作模式：正常、紧急、
 2. 峰值负载、过载
5. 响应：处理事件，改变服务等级
6. 响应度量：
 1. 延迟、逾期、吞吐量、抖动、积压率



常用的性能度量指标

指标	含义
延迟（Latency）	延迟系统产生反应所需要的时间。
逾期次数（Deadlines in processing）	在特定时间内完成某项任务的要求。
系统吞吐量（Throughput）	吞吐量是指系统在单位时间内能够处理的事务数量。
响应的抖动（Jitter）	响应的抖动是指响应时间的波动幅度。
事件积压	未处理的事件数量指的是由于系统过于繁忙无法及时响应而未能处理的事件。



02 复制
战略

战术概述

> 性能的影响:

处理时间
等待时间

对应两种战术

缩短等待时间的战术
缩短处理时间的战术

严禁复制

战术1-缩短等待时间-为什么会等待

- 请求太多
 - 系统支持100并发请求，超过100的请求会等待；
- 处理太慢
 - 每分钟到来100个请求，但是系统每分钟只能处理80个；
- 资源冲突
 - 两个请求都需要网络带宽，但是带宽只能支持一个请求；

禁止转载

战术1-缩短等待时间的战术



请求管理

降低请求数
动静分离
串行改并行



增加计算资源

增加单机容量
增加实例数量
(无技术含量)



减少冲突

悲观锁 VS 乐观锁
事务隔离级别
无锁队列
消除资源绑定

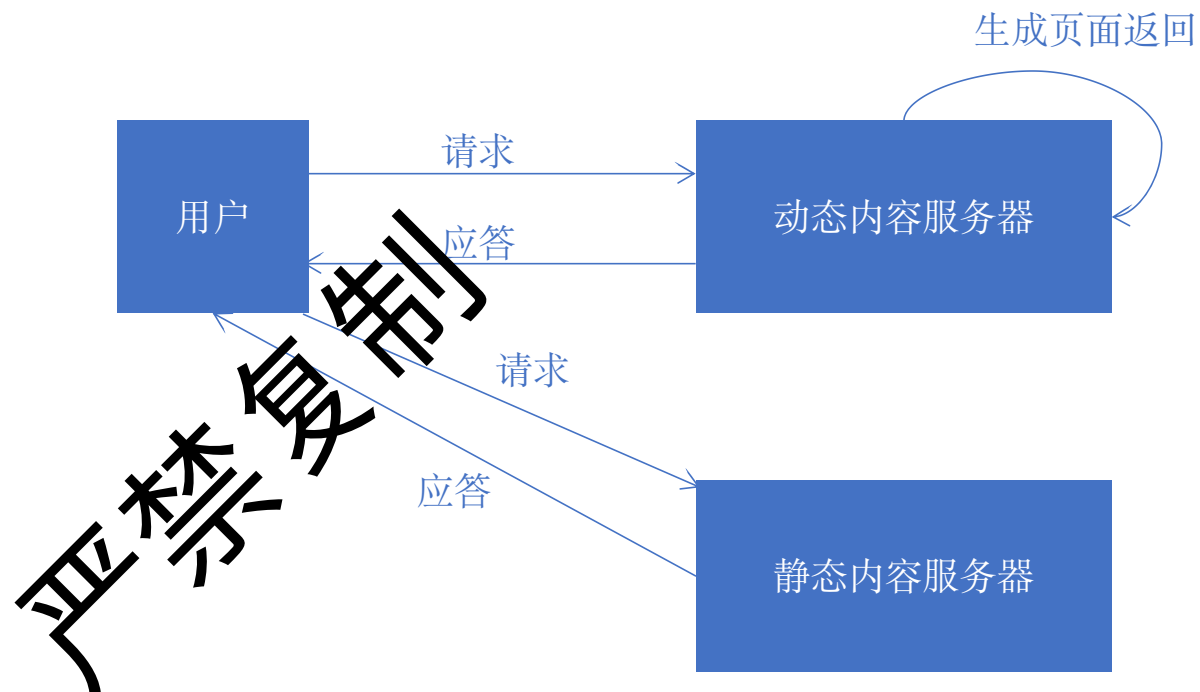
请求管理-降低请求数

- 定义：
 - 采样率：只处理一定百分比请求，反推出全局数据；
 - 无效请求：对用户无收益的请求；
- 调整采样率：
 - 审核类功能；
 - 监控类功能；
 - 对用户无直接影响的功能；
- 丢弃无效请求
 - 有限资源下的超长等待队列；
 - 不符合规范请求；

严禁复制

降低请求数-动静分离

- 动态内容:
 - 消耗资源;
 - 并发瓶颈;
 - 支持个性化;
- 静态内容:
 - 离线计算;
 - 0资源;
 - 无限并发;
 - 不支持个性化;



请求管理-串行改并行

- 效果:
 - 多个请求同时发送, 不必等待请求返回
- 原理:
 - JS中fetch, XMLHttpRequest等API
 - 浏览器并行处理多个request
 - HTTP/2,HTTP/3 的Multiplexing多路复用

禁止复制

减少冲突-悲观锁 VS 乐观锁

01

悲观锁:

假设冲突会发生，预防加锁；
加锁时间长；
等待概率高；
阻止无效修改；

严禁复制

02

乐观锁:

假设冲突不会发生，提交时检查；
无加锁等待的时间；
有一定的失败概率；
适用于冲突较少的情况；

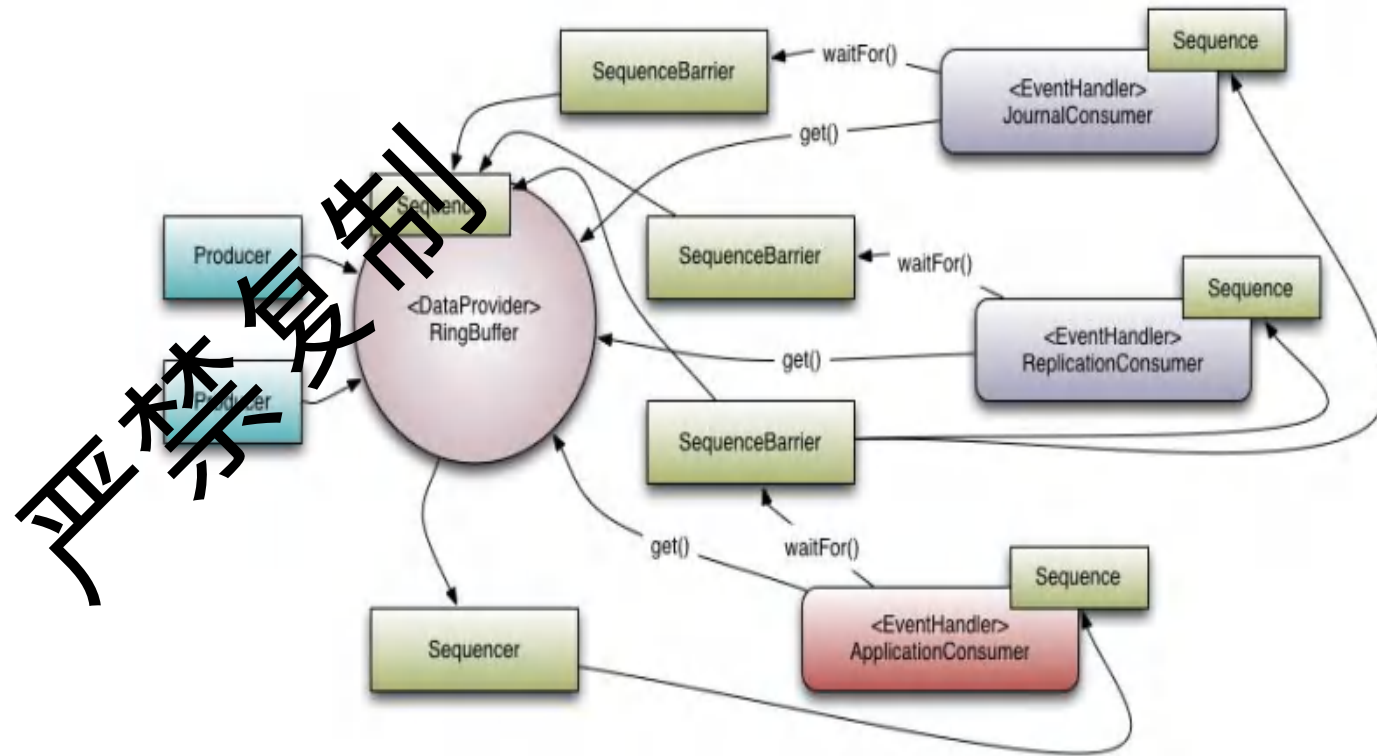
减少冲突-无冲突队列

- LMAX Disruptor 是一个高性能的线程间消息传递库。

- 功能类似于BlockingQueue
- 百万event每秒
- 无锁 (CAS)
- 预分配
- 支持多个consumer

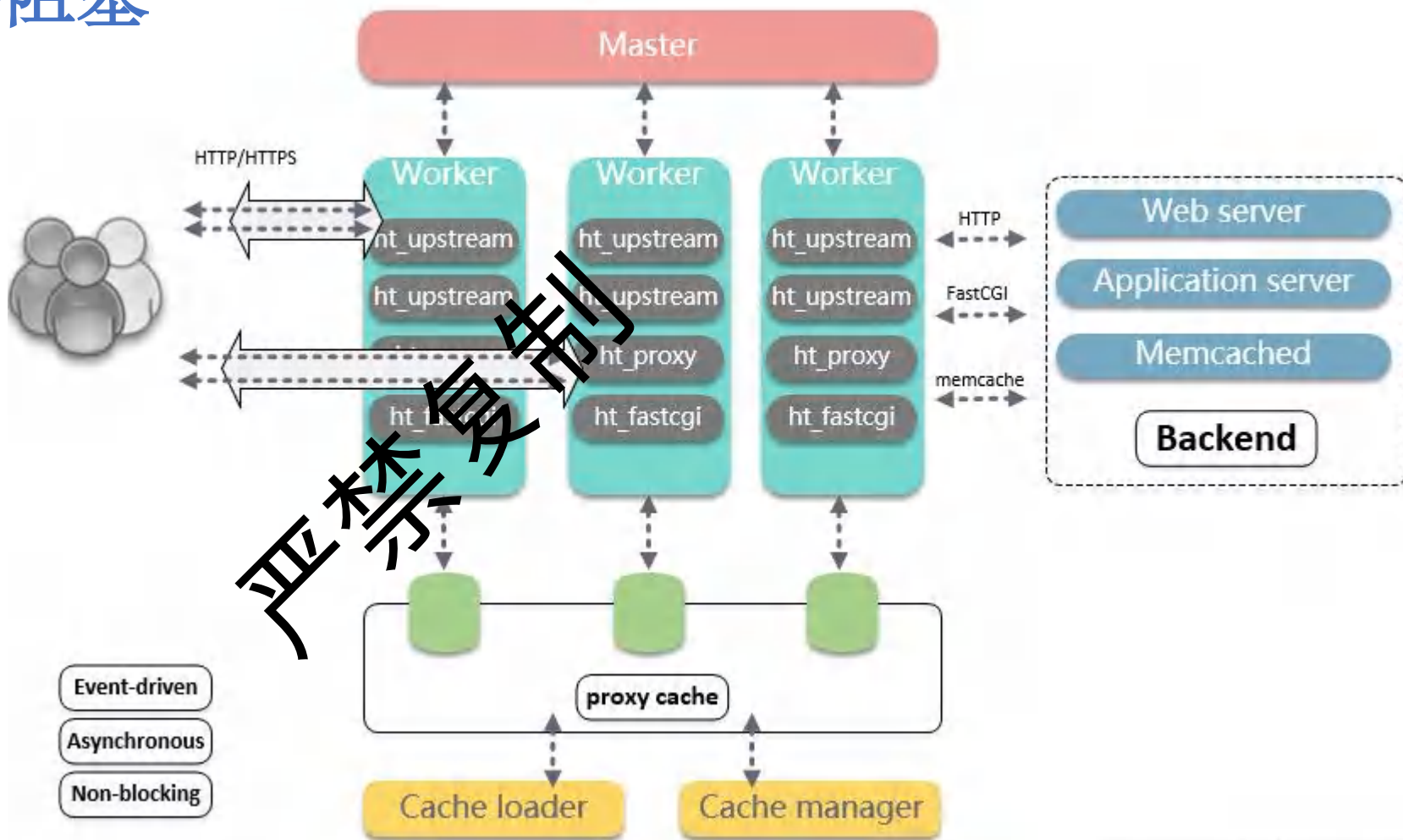
- 参考资料:

- <https://lmax-exchange.github.io/disruptor/user-guide/index.html>
- 【从BlockingQueue到无锁Disruptor的性能提升】
<https://cloud.tencent.com/developer/article/1490647>



减少冲突-消除IO阻塞

- nginx vs apache
 - qps 10万 VS 2万
- 原理
 - master调度worker
 - worker处理用户请求
 - 非阻塞IO
 - 事件通知机制



<https://nginx.org/en/>

战术2-缩短处理时间



减少计算量

加缓存
流式处理
读写分离



提高计算速度

并行计算
优化算法
复制
分区

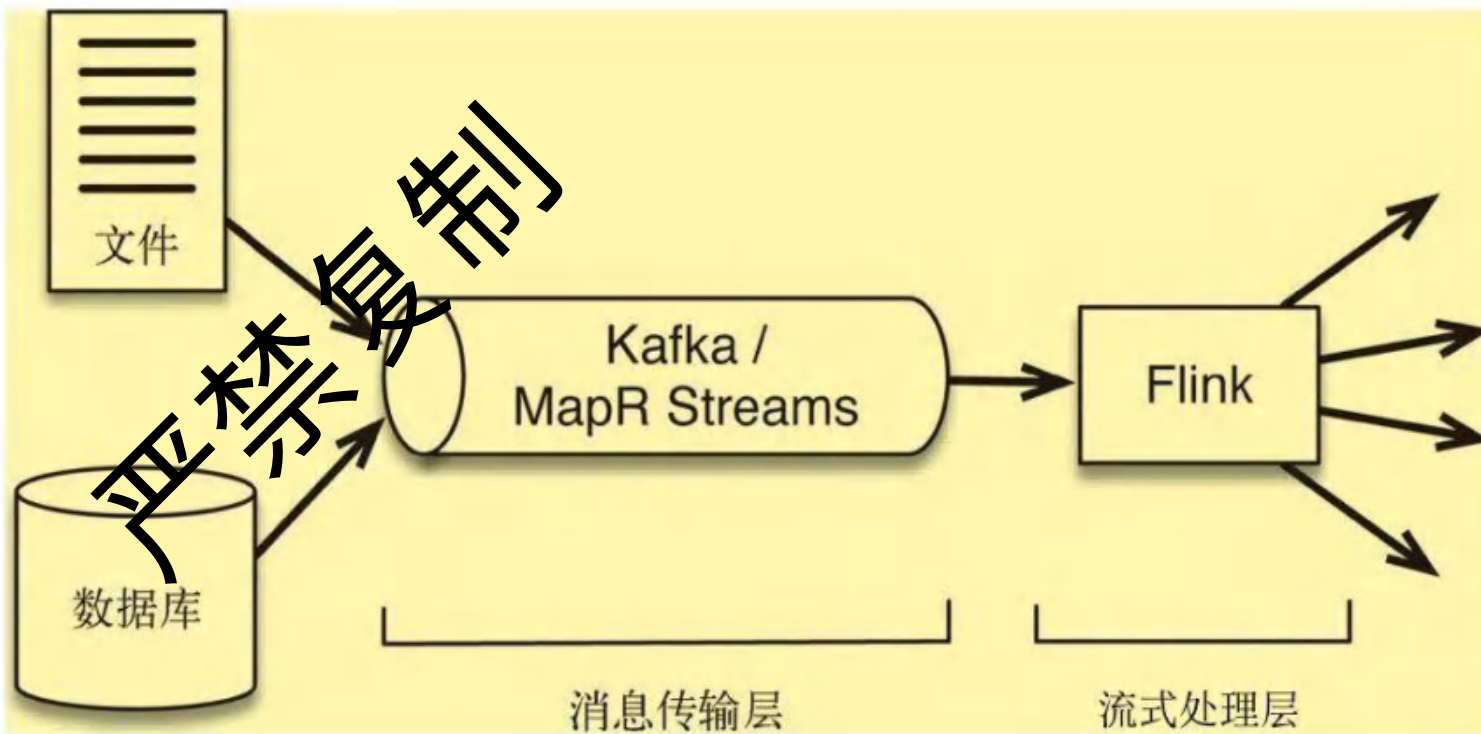


缩短查询时间

使用缓存
使用es查询

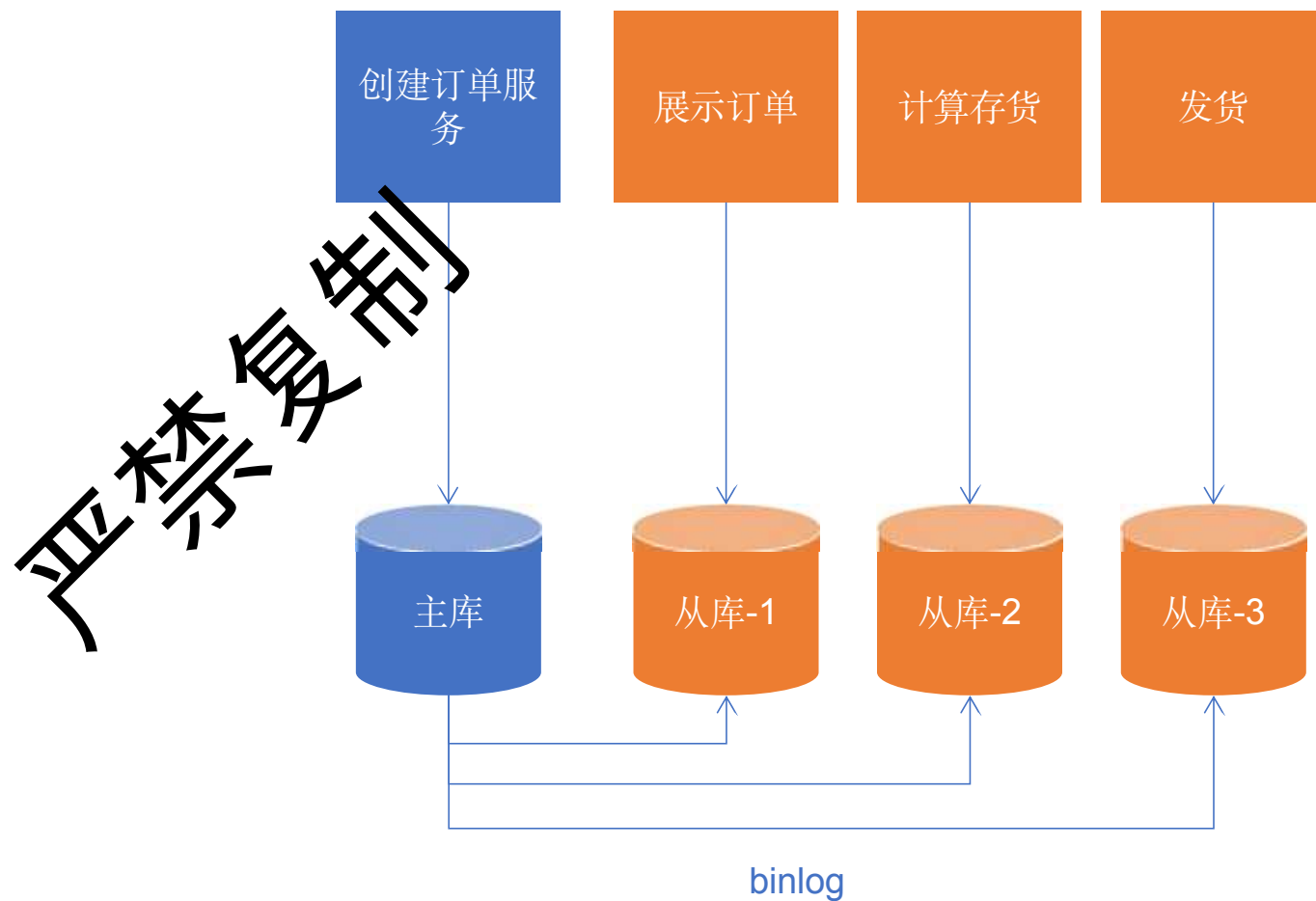
减量-流式处理减少计算量

- 处理流程：
 - 实时收集生产系统事件
 - 聚合/过滤/转化
- 实现效果
 - 之前：每秒上万条 → 每日处理上亿条（PB级）；
 - 之后：按分钟聚合 → 每日仅需处理约100万条（1%）
- 注意点
 - 网络延迟导致跨窗口到达



减量-读写分离减少计算量

- 实施方案
 - 主库负责写入
 - 读库负责查询
- 主从同步
 - log_bin
 - replication user
 - GTID (全局事务标志符)
 - IO和SQL线程



提速-全量复制分担流量

- 方法:

- 将数据复制到多个服务器，均匀分担计算负载，提高处理速度;
- 单leader、多leader、无leader
- 同步、异步

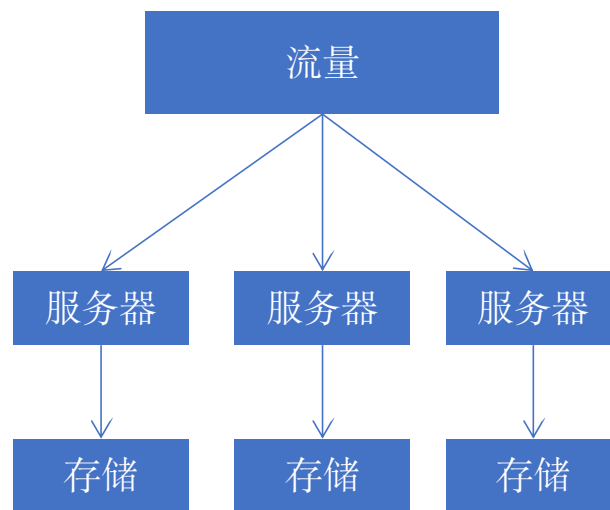
- 效果:

- 随着复制的数量，性能线性增长;

- 风险:

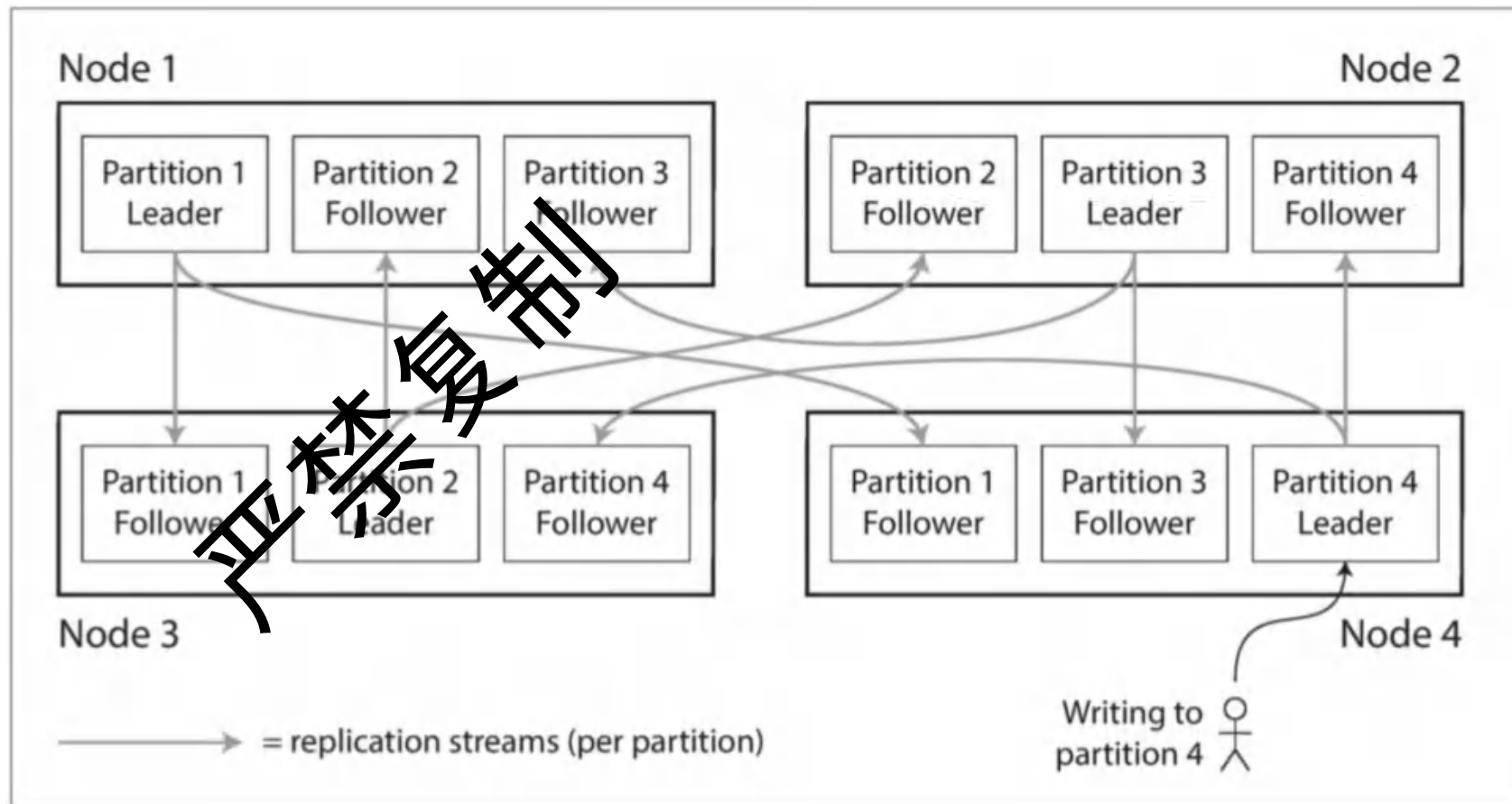
- 网络传输失败;
- 一致性: 读写延迟, monotonic (单调) 读, 一致性前缀

严禁复制

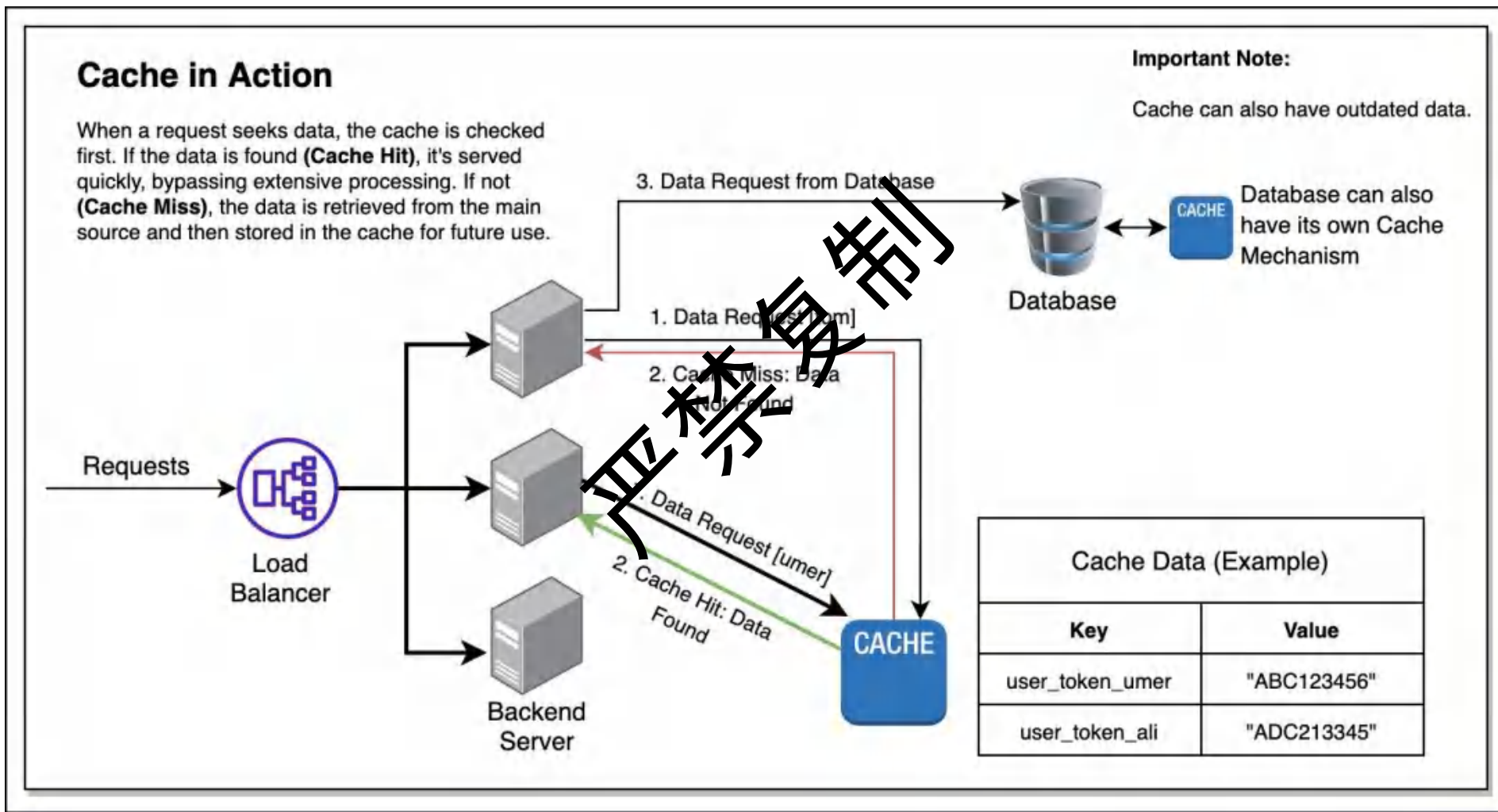


提速-多实例管理不同分区的数据

- 方法:
 - key-value分区
 - 固定分区、动态分区
 - 请求路由
- 效果
 - 分区数据量降低，性能提升
- 风险
 - 分区不均衡



提速：缓存



Level 1数据库 (RDS)

- 这一层通常不是指一个独立的“缓存产品”，
 - 缓存机制、数据库特性，减少对更深层次存储的访问。
 - 目标是让频繁访问的数据尽可能快地从数据库层面获取，避免磁盘I/O。
- 关系型数据库 (RDBMS) :
 - MySQL/MariaDB: InnoDB Buffer Pool, Query Cache , MyISAM Key Cache.
 - PostgreSQL: Shared Buffers, WAL (Write-Ahead Logging) 优化.
 - Oracle: System Global Area (SGA) 中的 Database Buffer Cache, Shared Pool.
 - SQL Server: Buffer Pool.
- 物化视图 (Materialized Views):
 - 预先计算并存储复杂查询的结果，可以看作是一种数据库级别的缓存。
- 只读副本 (Read Replicas): 将读操作分流到副本，减轻主库压力
 - 间接提升主库处理写操作和缓存未命中读操作的速度。

Level 1数据库 (Nosql)

- 不是独立的“缓存产品”
 - 目标是让频繁访问的数据尽可能快地从数据库层面获取，避免磁盘I/O。
- NoSQL 数据库：
 - MongoDB可以将热点数据集保持在内存中；
 - Dynamo 分布式无模式的key value存储；
 - Amazon DynamoDB Accelerator (DAX):
 - 专门为DynamoDB设计的全托管、高可用的内存缓存服务，可以大幅提升读取性能。
 - HBase通过自身的缓存机制（MemStore 和 BlockCache）来提高处理速度
 - Cassandra:
 - 高度可扩展、高可用的分布式 NoSQL 数据库。
 - 它的核心功能是持久化存储大量数据，并提供低延迟的读写访问。
- Elasticsearch索引服务：
 - 内部有多种缓存机制（如Field Data Cache, Query Cache, Request Cache）

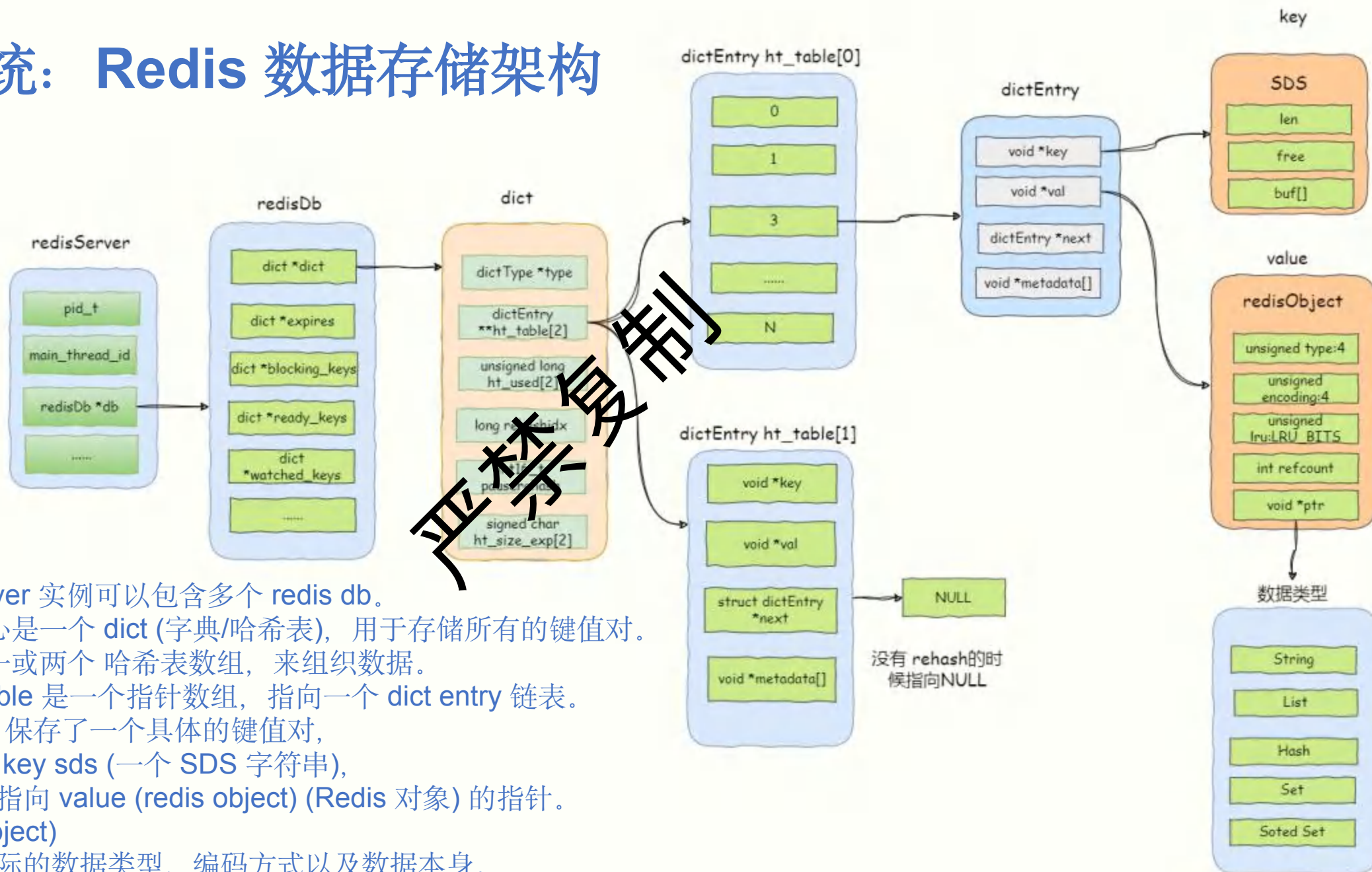
Level 2 分布式缓存 (out of process)

- 独立于应用和数据库的缓存服务，
 - 通常部署在多台服务器上，形成一个集群。它为多个应用实例提供共享的高速缓存。
- 可供选择的产品：
 - **Redis**: 非常流行的高性能键值存储，支持多种数据结构，持久化，主从复制，哨兵和集群模式。
 - **Memcached**: 另一个广泛使用的内存键值存储，简单高效，专注于缓存。
 - **Hazelcast**: 开源的内存数据网格 (MDG)，提供分布式数据结构、计算和消息传递。
 - **Apache Ignite**: 内存计算平台，提供分布式缓存、数据库和计算能力。
 - **Amazon ElastiCache**: AWS提供的托管式Redis或Memcached服务。
 - **Google Cloud Memorystore**: GCP提供的托管式Redis或Memcached服务。
 - **Azure Cache for Redis**: Azure提供的托管式Redis服务。

Level 3 内存缓存

- 这一层缓存数据直接存储在应用程序的进程内存中，访问速度最快，因为没有网络开销。但它的缺点是缓存数据不共享（每个应用实例有自己的副本），且随应用进程生命周期结束而消失。
- 可供选择的产品/库：
- Java:
 - Caffeine: 高性能的Java 8+本地缓存库。
 - Google Guava Cache: Google提供的强大大本地缓存实现。
 - Ehcache (v3+可以作为进程内缓存): 老牌Java缓存框架。
- Python:
 - `functools.lru_cache` / `functools.cache` (Python 3.9+): Python标准库提供的简单LRU缓存装饰器。
 - `cachetools`: 一个可扩展的Python缓存库，支持多种缓存策略。

关键系统：Redis 数据存储架构



缓存使用的挑战 (redis及其它)

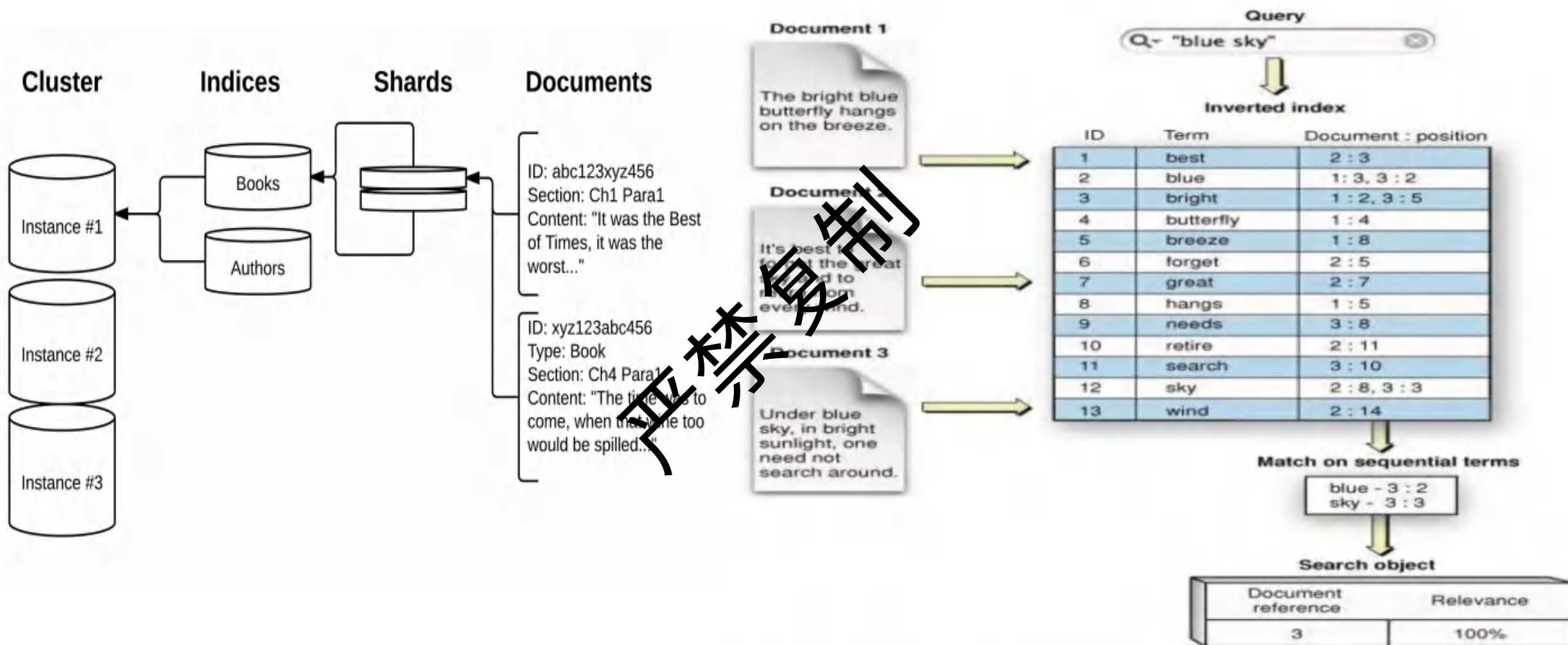
1. 数据一致性 (Cache Coherency / Data Staleness):
 1. 主动更新, 旁路缓存, read-through
2. 缓存穿透 (Cache Penetration):
 1. 访问一个不存在的数据, 缓存null, bloom filter
3. 缓存雪崩 (Cache Avalanche):
4. 缓存击穿 (Cache Breakdown / Hotspot Invalidation):
 1. 热点数据过期, 分布式锁
5. 缓存预热 (Cache Warm-up):
6. 缓存的内存管理与大小限制:
 1. LRU, LFU, FIFO
7. 读写延迟
8. 序列化与反序列化开销

严禁复制

ES的高性能的原因

- 基石：倒排索引 (Inverted Index)
 - 机制：“词元 -> 文档列表”，而非“文档 -> 词元”。
 - 流程：文档分词 -> 构建词典 -> 生成含位置/频率的发布列表。
 - 优势：直接定位含特定词元的文档，无需全表扫描。
- 结构：文档型数据库 (Document-Oriented)
 - 特点：JSON 格式，数据聚合存储。
 - 优势：避免昂贵的 Join 操作，查询路径短。
- 引擎：Lucene 底层优化
 - 技术：高效数据结构 (如 FST)、压缩算法、快速查找/匹配算法 skip lists, bitsets。
 - 优势：空间占用小，查找效率高。
- 加速：多级缓存机制 (Caching)
 - 层面：节点查询缓存、分片请求缓存、OS 文件系统缓存、字段数据缓存。
 - 优势：大幅减少重复计算和磁盘 I/O，命中缓存则极速返回。
- 架构：分布式设计 (Distributed)
 - 核心：索引分片 (Shards) + 并行处理。
 - 优势：查询任务分解到多节点并行执行，结果聚合，充分利用集群资源。
- 特性：列式存储 (Doc Values)
 - 场景：专为排序、聚合优化。
 - 优势：按列组织数据，仅访问所需字段，极大提升分析效率。

ES架构：分区+倒排索引





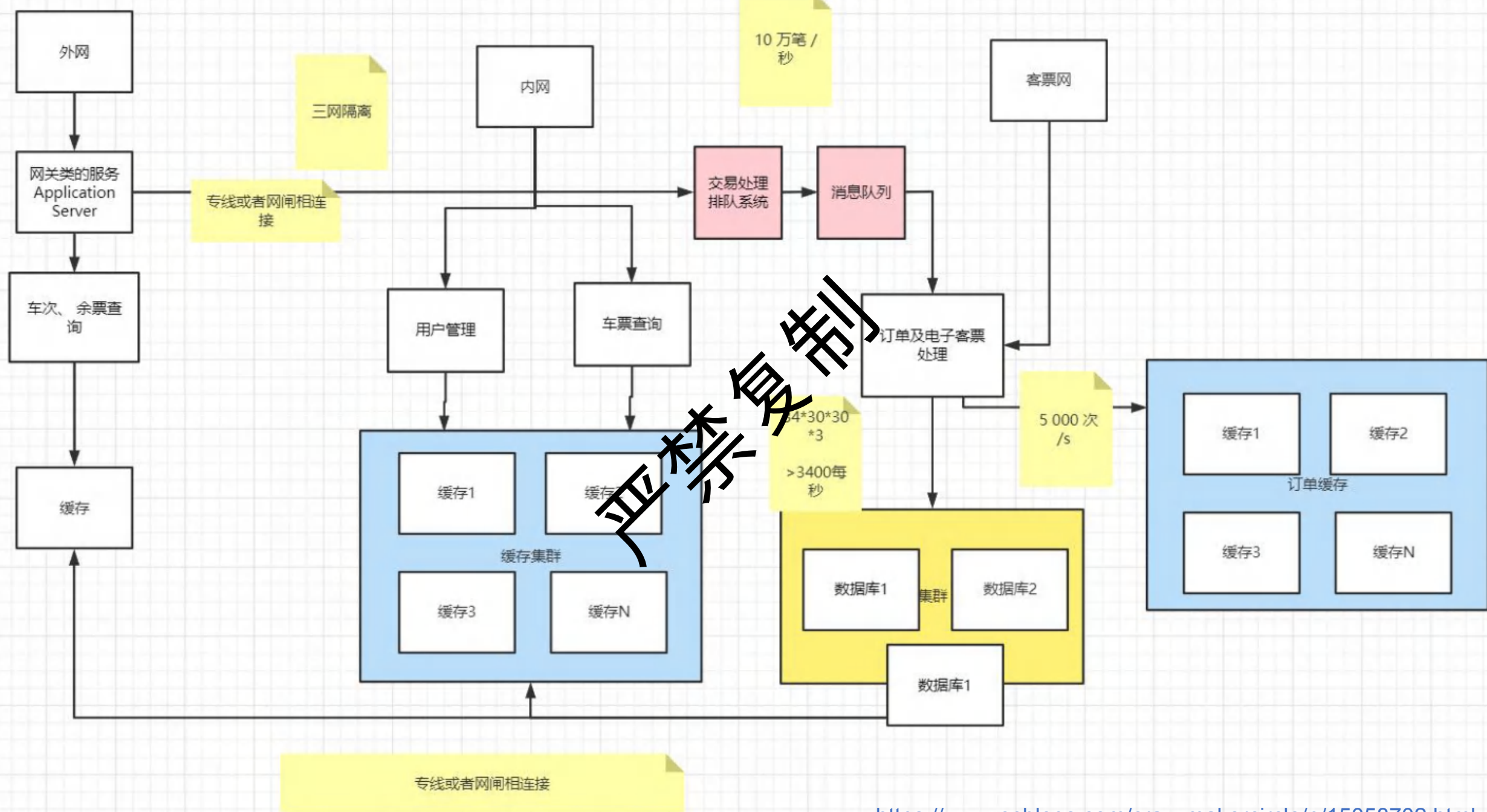
03 复制案例 案例分析

12306的案例分析

- 12306的意义
 - 社会收益：公平
 - 个体收益：幸福感
 - 国家收益：增加国家收入
- 技术方案
 - 缓存
 - 分区
 - 队列
 - 候补
 - 预定

严禁复制







04 复制 结构化决策

架构决策

- 容量规划：
 - 系统预期的请求量，计算量，数据量。
- 负载均衡：
 - 流量的合理分配和调度，避免单点瓶颈。
- 数据库和缓存策略：
 - 数据库的读写负载，查询类型，查询性能，缓存的key-value设计，过期时间
- 职责分离：
 - 不同的计算职责分配不同的模块；
 - 使用不同的技术（CPU+GPU+FPGA）

第6课

伸缩性 易用性 互操作性 可修改性

汇报人：软件学院

严禁复制



01 复制
严格复制性（扩展性）

伸缩性

01

定义

增加资源是否能带来某些质量属性（例如性能）的提升；

02

两种实现方式：水平扩展（Scale Out） vs. 垂直扩展（Scale Up）

水平扩展：通过增加机器数量提升性能（如分布式集群），灵活性高，成本可控。

垂直扩展：提升单机性能（如升级CPU/内存），成本高且存在物理上限。

现代架构优先选择水平扩展。

03

可扩展性的关键指标

线性扩展能力：资源增加**N**倍，性能提升接近**N**倍。

弹性伸缩：根据负载动态调整资源（如突发流量自动扩容）。

跨地域扩展：支持全球分布式部署（如**CDN**、多活数据中心）。

设计高可扩展性架构的六大核心原则

无状态服务

01

服务不保存本地状态：可由任意节点处理。
实现方式：会话状态外置（如**Redis**存储**Session**）。业务逻辑与数据解耦。

分层与解耦

02

前后端分离：前端静态资源部署（**CDN**），
后端动态**API**独立扩展。

数据分片：

03

水平分片，垂直分片

消息队列：

04

削峰填谷，解耦依赖

微服务与容器化：

05

服务拆分，容器化部署

缓存多级化

06

客户端缓存 → **CDN** → 分布式缓存
(**Redis**) → 数据库缓存 (**Memcached**)



02 复制
易用性

易用性定义

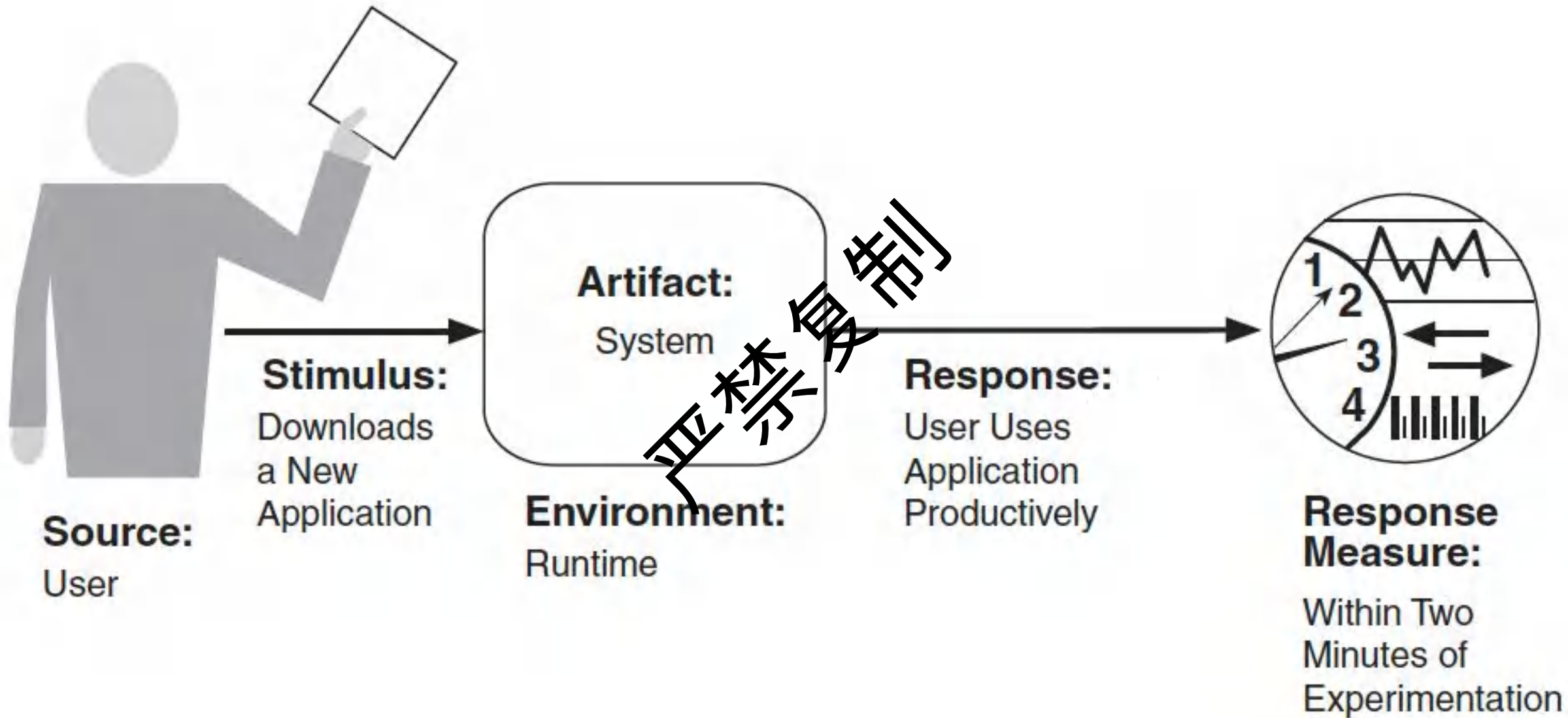
刺激因素：最终用户希望

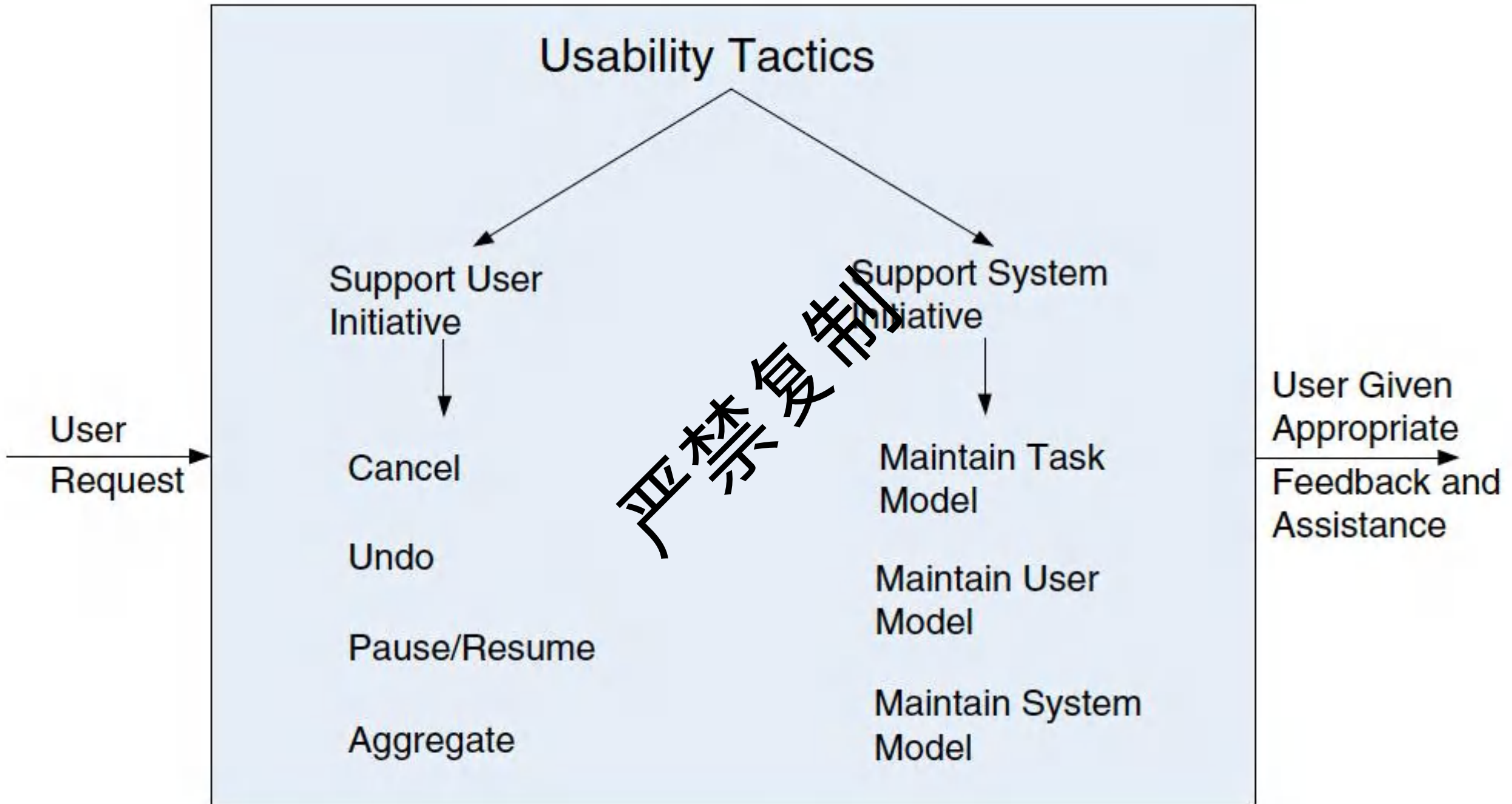
学会如何高效地使用系统、
最小化错误的影响、
配置定制化系统、系统自适应。

响应通过以下指标进行衡量：

结果指标：完成任务耗时、遇到的错误数量、完成任务的数量；
过程指标：成功操作，总操作，错误导致的时间，错误导致的数据；
用户指标：用户满意度、用户知识的获得。

严禁复制





系统需要支持4种用户操作

取消操作(Cancel):

用户在操作过程中终止正在执行的任务。
比如，“取消”按钮停止文件下载。

撤销操作(Undo):

让用户撤回之前的操作，
比如：撤销最近一次的文本修改。
技术：快照，可逆操作

暂停/恢复操作(Pause/Resume):

对于需要较长时间的任务，
比如：用户暂停视频然后继续。
技术方案：保存信息

批量操作(Aggregate):

当用户需要对多个对象执行相同操作时，
比如：统一字体大小，可以避免重复劳动。

严禁复制

系统采取行动（1）基本概念

› 维护任务模型

用于帮助系统理解用户正在做什么,从而提供相应的帮助。

› 维护用户模型

代表用户对系统的理解、行为模式和其他特定信息,可以调整系统的反应。

› 维护系统模型

系统自我了解自己的状态,以便提供恰当的反馈。

系统采取行动（2）利用模型做出预测

任务模型：

1. 任务模型帮助系统了解上下文,即用户正在执行什么任务。
2. 通过了解任务背景,系统能够提供有针对性的帮助。

用户模型：

1. 用户模型用于记录用户的知识和行为,例如: 熟悉系统的程度、行为习惯(预期响应时间)。
2. 系统可以自动调整参数。
例如,鼠标选择的速度, 避免一不小心选中整个文档。

系统模型：

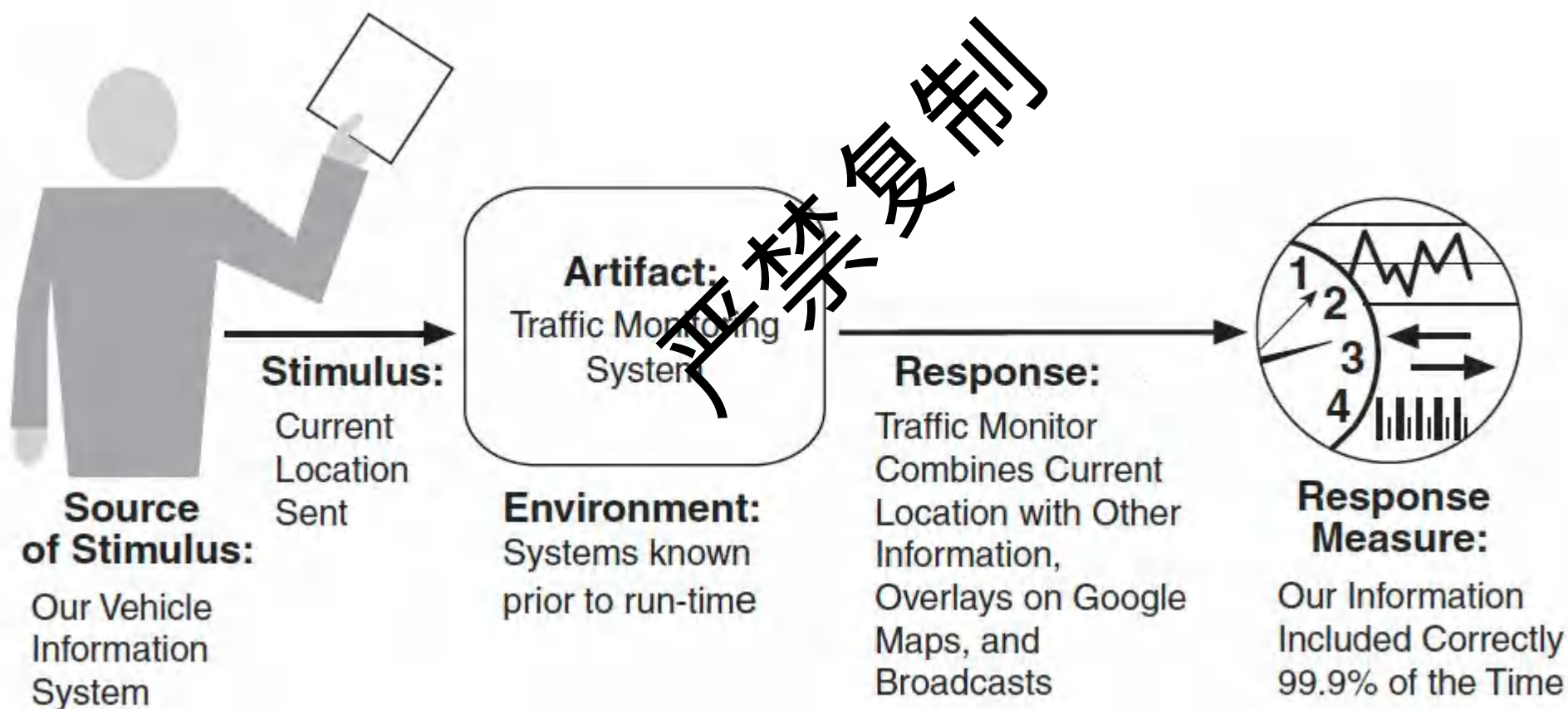
1. 系统了解自身的状态, 预测自己可能的行为, 并作出相应的反馈。
2. 例如,系统可以通过进度条告诉用户任务完成的预计时间,从而让用户知道还需要等多久。



03 复制性 可操作性 严格复制

互操作性的定义

互操作性：两个系统在特定的context中互相正确的传递数据，并且能够正确的理解数据的能力。



互操作性的影响

系统集成：多个系统能不能
整合成一个整体；



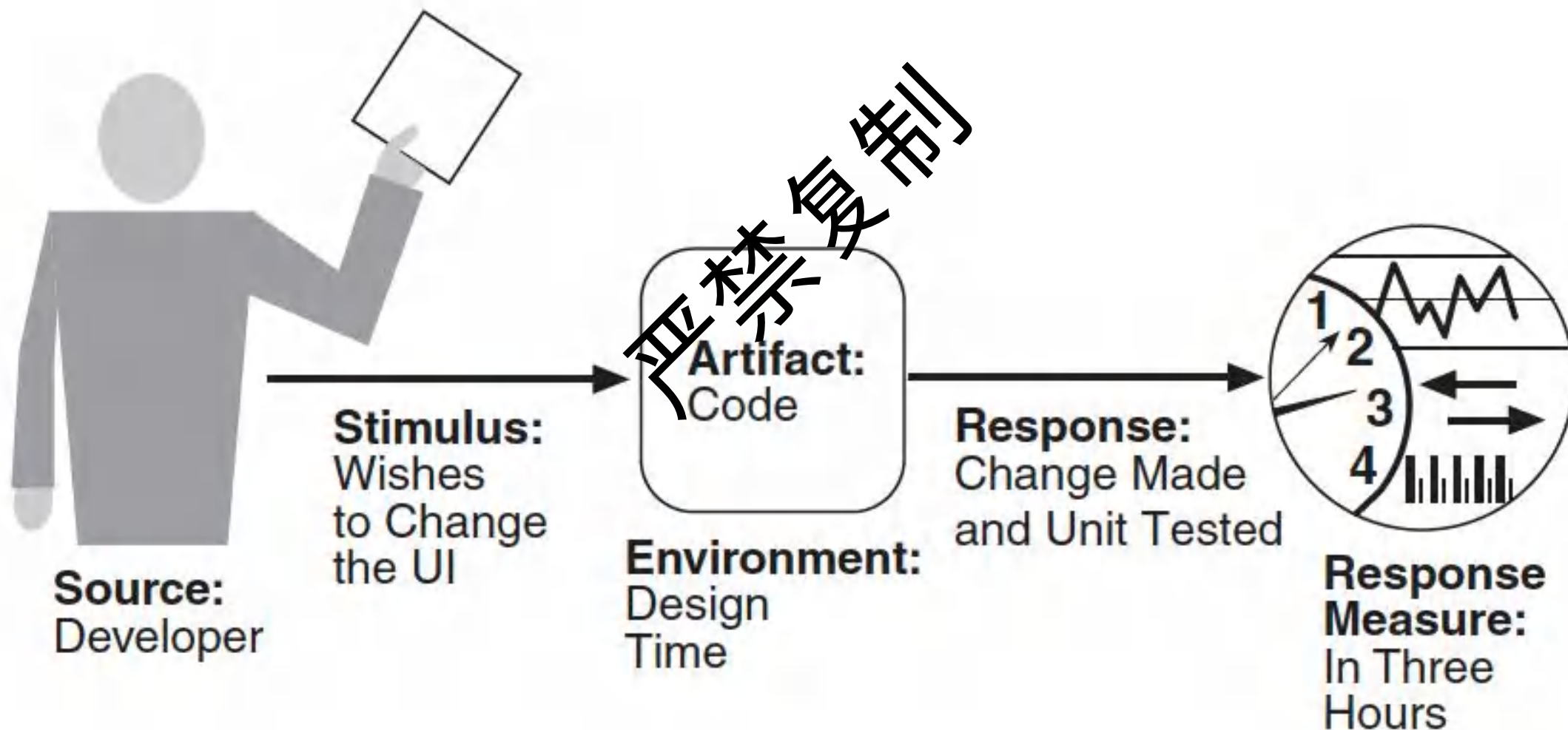
服务稳定性：当上游接口发
生变化时，系统能否兼容；

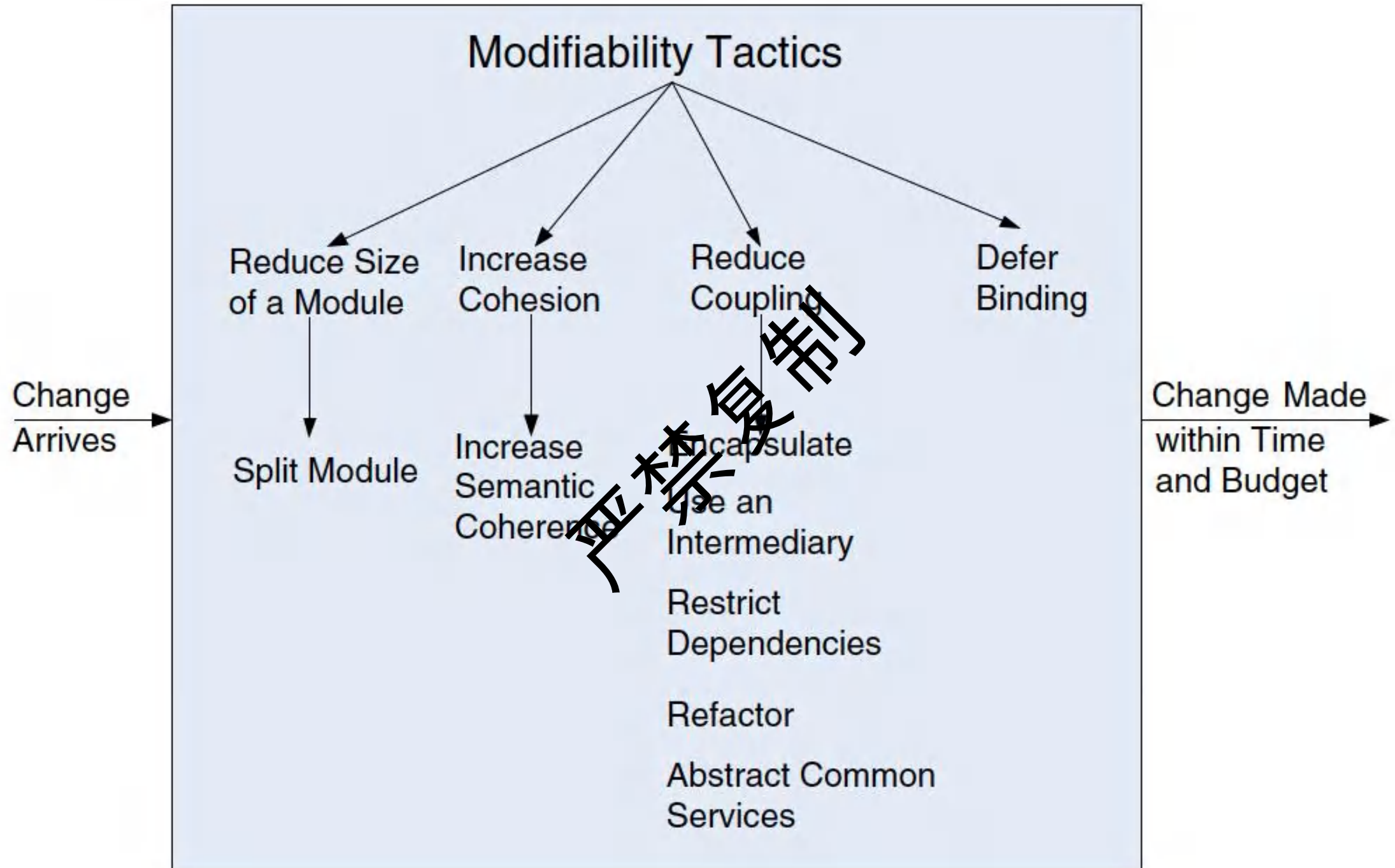


04 复制
可修改性（灵活性）
严格控制

可修改性的场景描述

- 可修改性：完成特定修改所需要的资源数量。





作业

- 在上一次的架构文档的基础上，补充“10 质量需求”。

10. 质量需求

(回顾第 1 节的目标)

- **性能:** 物理计算 (系统模块) 和渲染 (显示模块) 是性能关键路径，需要优化算法和数据结构。
- **精度:** 物理模拟需要足够的精度来提供可信的驾驶感受，但不必完全仿真。
- **可测试性:** 模块化设计有助于单元测试（如测试 道路模块 的查询，系统模块 的物理计算）。



严禁复制

Thank you

感谢观看

汇报人：北交大软件学院 ——